

Study Report: Full Stack Web Developer - Career Path & Core Skills

Written by Gagan Pasupuleti

Family:	Full Stack Web Developer
Level:	Beginner to Advanced
Tracks:	Beginner, Intermediate, Advanced
Chapters:	18
Source:	Official docs, free libraries, and industry-standard references (see sources and books)

Official sources and free libraries

- <https://developer.mozilla.org/en-US/docs/Web>
- <https://react.dev/>
- <https://expressjs.com/>
- <https://www.postgresql.org/docs/>
- <https://nodejs.org/docs/latest/api/>
- <https://jwt.io/introduction>

Summary

End-to-end web development spanning HTML/CSS/JavaScript, React frontends, Node.js/Express APIs, PostgreSQL persistence, authentication, and cloud deployment. Builds complete products from database schema to deployed SPA with secure auth and CI pipelines.

Skill stack by level

Beginner

- HTML5 structure and CSS responsive layouts
- JavaScript DOM manipulation and fetch API
- Git branching and pull request workflow
- Node.js basics and npm scripts
- Express routing and middleware
- PostgreSQL tables, SELECT, INSERT, JOIN
- Environment variables and .env patterns

Intermediate

- React SPA with component architecture
- REST API design and status code semantics
- SQL migrations and ORM/query builders (Prisma/Drizzle)

- Session and JWT authentication flows
- CORS, cookies, and secure HTTP headers
- Input validation on client and server
- Docker Compose full-stack local dev
- Basic AWS/Vercel/Railway deployment

Advanced

- OAuth social login integration
- WebSockets or SSE for real-time features
- Caching with Redis
- File uploads to S3-compatible storage
- Rate limiting and API security hardening
- Monorepo tooling (Turborepo)
- Production monitoring and error tracking (Sentry)

Recommended books

1. Eloquent JavaScript - Marijn Haverbeke

Level: Beginner | JavaScript, DOM, Node.js - free interactive online book.

2. Full Stack React and TypeScript - Ryan Lanciaux

Level: Intermediate | React frontend with TypeScript and API integration patterns.

3. Node.js Design Patterns - Mario Casciaro & Luciano Mammino

Level: Intermediate | Express APIs, middleware, streams, and scalable Node architecture.

4. PostgreSQL: Up and Running - Regina Obe & Leo Hsu

Level: Intermediate | Relational schema design, queries, and indexes for web apps.

5. The Pragmatic Programmer - David Thomas & Andrew Hunt

Level: Advanced | Professional habits: DRY, testing, automation, and craft.

End-to-end projects

Project 1: Todo App Full Stack

Beginner | 2 weeks - React frontend + Express API + SQLite: create, complete, delete todos with persistence.

- API: API tested with curl
- UI: Working frontend
- Connect: Full stack running
- Deploy: Live URLs

Project 2: CodeQuest Teacher Review Platform

Intermediate | 4-5 weeks - Teachers log in, browse reports by family, mark rubric scores, download PDF - full auth stack.

- Auth: Auth Postman tests
- Reports: Reports browse page
- Marking: Marking workflow demo
- Deploy: Production URL

Project 3: SaaS Subscription Platform

Advanced | 6-8 weeks - Multi-tenant SaaS with Stripe billing, team invites, admin panel, and observability.

- Billing: Test subscription flow
- Tenants: Multi-tenant tests
- Teams: Team management UI
- Ops: Ops runbook

Chapters

Track: Beginner

Chapter 1: Full Stack Developer Role Map [Beginner]

Chapter focus: Full Stack Developer Role Map** *(Level: Beginner)

Full stack developers own features from database to UI, coordinating across layers. You understand how HTTP requests flow from browser to server to database and back as JSON. Breadth is your strength-depth grows in frontend or backend specialization over time. Modern stacks pair React SPAs with Node APIs and PostgreSQL for relational data.

Code Reference:

```
// Client
const res = await fetch('/api/tasks');
const tasks = await res.json();

// Server (Express)
app.get('/api/tasks', async (req, res) => {
  const { rows } = await pool.query('SELECT * FROM tasks ORDER BY id');
  res.json(rows);
});
```

What it shows: fetch calls same-origin API; Express queries Postgres and returns JSON array.

Topic guide

What it actually is

Full stack development spans client, server, database, and deployment layers.

When to use it

Choose when you want ownership of entire features and rapid prototyping ability.

Where to use it

Startups, agencies, indie products, and small product teams.

Real use example

Solo founder ships MVP in weeks because one person wires UI, API, and DB together.

Chapter 2: HTML/CSS/JS Browser Foundations [Beginner]

Chapter focus: HTML/CSS/JS Browser Foundations *(Level: Beginner)**

The browser parses HTML into DOM tree, applies CSS cascade, and executes JavaScript. Document Object Model APIs let scripts read and mutate page content dynamically. Event delegation handles clicks efficiently on lists rendered from data. fetch replaces legacy XMLHttpRequest for HTTP from the browser.

Code Reference:

```
document.querySelector('#add-btn').addEventListener('click', async () => {
  const title = document.querySelector('#title').value;
  await fetch('/api/tasks', { method: 'POST', headers: { 'Content-Type':
'application/json' }, body: JSON.stringify({ title }) });
  loadTasks();
});
```

What it shows: Click handler reads input, POSTs JSON, then reloads task list.

Topic guide**What it actually is**

Browser APIs connect user interactions to backend services via HTTP.

When to use it

Master before adopting React-concepts transfer directly to component event handlers.

Where to use it

Legacy pages, lightweight widgets, and understanding framework abstractions.

Real use example

Todo widget on marketing site POSTs new items without full SPA framework.

Chapter 3: Node.js and Express Server Basics [Beginner]**Chapter focus: Node.js and Express Server Basics** *(Level: Beginner)**

Node.js runs JavaScript on the server with non-blocking I/O event loop. Express adds routing and middleware chain for HTTP request processing. Middleware parses JSON bodies, logs requests, and handles errors uniformly. Organize routes in modules as API surface grows.

Code Reference:

```
const express = require('express');
const app = express();
app.use(express.json());
app.get('/api/health', (req, res) => res.json({ status: 'ok' }));
app.listen(3000);
```

What it shows: express.json() parses POST bodies; health route confirms server running.

Topic guide**What it actually is**

Express is the minimal web framework for Node.js REST APIs.

When to use it

Use for JSON APIs serving React SPAs and mobile clients.

Where to use it

CRUD backends, webhooks, and server-rendered hybrids.

Real use example

Mobile app consumes Express JSON API hosted on same domain with reverse proxy.

Chapter 4: PostgreSQL Fundamentals for Web Apps [Beginner]

Chapter focus: PostgreSQL Fundamentals for Web Apps *(Level: Beginner)**

PostgreSQL stores relational data with ACID transactions and rich SQL features. Primary keys identify rows; foreign keys enforce referential integrity between tables. JOIN combines related tables-users with their posts in one query. Indexes speed lookups but slow writes-add for frequent WHERE columns.

Code Reference:

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

```
SELECT u.email, p.title FROM users u JOIN posts p ON p.user_id = u.id;
```

What it shows: SERIAL auto-increments id; JOIN links posts to owners by foreign key.

Topic guide

What it actually is

PostgreSQL is the default relational database for modern web applications.

When to use it

Use when data has relationships, transactions, or complex queries.

Where to use it

User accounts, orders, content management, and analytics warehouses.

Real use example

Dashboard query JOINS orders and customers for revenue-by-segment report.

Chapter 5: Connecting Express to PostgreSQL [Beginner]

Chapter focus: Connecting Express to PostgreSQL *(Level: Beginner)**

node-postgres (pg) Pool reuses connections efficiently under concurrent requests. Parameterized

queries (\$1, \$2) prevent SQL injection-never interpolate user strings. Transactions wrap multiple statements in atomic commit or rollback. Handle pool errors and close gracefully on SIGTERM during deploys.

Code Reference:

```
const { rows } = await pool.query(
  'INSERT INTO tasks (title) VALUES ($1) RETURNING *',
  [req.body.title]
);
res.status(201).json(rows[0]);
```

What it shows: \$1 placeholder binds title safely; RETURNING sends created row to client.

Topic guide

What it actually is

Database pools bridge Express handlers and PostgreSQL with safe parameterized SQL.

When to use it

Wire up on every Express project using raw SQL or ORM underneath.

Where to use it

API routes persisting user-generated content.

Real use example

Create task endpoint inserts row and returns 201 with new id in milliseconds.

Chapter 6: Git Workflow and Project Structure [Beginner]

Chapter focus: Git Workflow and Project Structure** *(Level: Beginner)

Feature branches isolate work; pull requests enable review before merging to main. Monorepos place client/ and server/ in one repository with shared tooling. Separate repos suit teams with independent release cycles. Consistent README, .env.example, and scripts (dev, test, build) onboard teammates fast.

Code Reference:

```
# package.json scripts
"scripts": {
  "dev": "concurrently \"npm run dev:server\" \"npm run dev:client\"",
  "dev:server": "nodemon server/index.js",
  "dev:client": "vite"
}
```

What it shows: concurrently runs API and Vite dev servers with one npm run dev command.

Topic guide

What it actually is

Structured repos and Git workflows enable team collaboration on full stack apps.

When to use it

Establish conventions before second contributor joins.

Where to use it

Every professional web project and open-source full stack template.

Real use example

New contributor clones repo, copies .env.example, runs npm dev successfully.

Track: Intermediate

Chapter 7: React SPA Consumption of REST APIs [Intermediate]

Chapter focus: React SPA Consumption of REST APIs *(Level: Intermediate)**

SPAs fetch JSON after initial HTML shell loads; routing happens client-side. Centralize API calls in services/hooks with base URL from environment. Handle 401 globally-redirect to login when token expires mid-session. Optimistic UI updates feel fast; rollback on server rejection.

Code Reference:

```
async function createTask(title) {
  const res = await fetch(`${API_URL}/tasks`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', Authorization: 'Bearer ${token}' },
    body: JSON.stringify({ title }),
  });
  if (!res.ok) throw new ApiError(res.status, await res.text());
  return res.json();
}
```

What it shows: Authorization header sends JWT; ApiError preserves status for UI handling.

Topic guide**What it actually is**

Client API layer abstracts HTTP details from React components.

When to use it

Build once and reuse across components and React Query hooks.

Where to use it

Dashboards, forms, and infinite scroll lists backed by Express.

Real use example

Task list optimistically adds card then rolls back if server returns 409 duplicate.

Chapter 8: Authentication with JWT and Cookies [Intermediate]

Chapter focus: Authentication with JWT and Cookies *(Level: Intermediate)**

Register stores bcrypt hash; login verifies and returns short-lived access JWT. HttpOnly cookies store refresh tokens safe from XSS JavaScript access. SameSite=Lax or Strict mitigates CSRF on cookie-based auth. Server middleware verifies JWT on protected routes before business logic.

Code Reference:

```
function authMiddleware(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ error: 'Unauthorized' });
  try { req.user = jwt.verify(token, process.env.JWT_SECRET); next(); }
  catch { res.status(401).json({ error: 'Invalid token' }); }
}
```

What it shows: Middleware extracts bearer token, verifies signature, attaches user payload.

Topic guide**What it actually is**

JWT auth enables stateless API authentication for SPAs.

When to use it

Implement on any app with user accounts and protected resources.

Where to use it

SaaS apps, social platforms, and admin tools.

Real use example

Expired access token triggers silent refresh via HttpOnly cookie endpoint.

Chapter 9: Schema Migrations with Prisma or SQL [Intermediate]**Chapter focus: Schema Migrations with Prisma or SQL** *(Level: Intermediate)**

Migrations version database schema alongside application code in Git. Prisma schema defines models generating type-safe client and migration SQL. Never edit production DB manually-always apply tested migration files. Rollback strategy: write reversible migrations or restore from backup.

Code Reference:

```
-- migration: 20260301_add_posts.sql
CREATE TABLE posts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  body TEXT NOT NULL
);
```

What it shows: UUID primary key; ON DELETE CASCADE removes posts when user deleted.

Topic guide

What it actually is

Migrations evolve database schema predictably across environments.

When to use it

Adopt before team collaboration or production deployment.

Where to use it

Every serious PostgreSQL-backed web application.

Real use example

Deploy pipeline runs prisma migrate deploy before starting new server version.

Chapter 10: CORS, Security Headers, and Validation [Intermediate]

Chapter focus: CORS, Security Headers, and Validation *(Level: Intermediate)**

CORS headers allow browser on localhost:5173 to call API on localhost:3000 during dev. Helmet sets secure defaults: X-Content-Type-Options, frameguard, HSTS in production. Validate req.body with Zod on server-never trust client validation alone. Rate limit auth endpoints to slow credential stuffing attacks.

Code Reference:

```
app.use(cors({ origin: process.env.CLIENT_URL, credentials: true }));
app.use(helmet());
app.use('/api/', rateLimit({ windowMs: 15 * 60 * 1000, max: 100 }));
```

What it shows: cors whitelists frontend origin; rateLimit caps requests per IP per window.

Topic guide**What it actually is**

HTTP security layers protect APIs consumed by browser-based clients.

When to use it

Configure before exposing any API to the public internet.

Where to use it

Production SPAs, OAuth callbacks, and cookie-based sessions.

Real use example

Pen test finds no open CORS * because only production domain is whitelisted.

Chapter 11: Docker Compose Full Stack Dev [Intermediate]

Chapter focus: Docker Compose Full Stack Dev *(Level: Intermediate)**

Compose defines api, db, and optional redis services in one YAML file. Named volumes persist Postgres data across container restarts. Development mounts source code for hot reload inside containers. Healthcheck on db prevents API starting before Postgres accepts connections.

Code Reference:

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: dev
  api:
    build: ./server
    depends_on:
      db:
        condition: service_healthy
```

What it shows: depends_on with health condition orders startup correctly.

Topic guide**What it actually is**

Docker Compose reproduces multi-service stacks on any developer machine.

When to use it

Use when stack has database plus one or more app services.

Where to use it

Local dev parity, onboarding, and integration test environments.

Real use example

QA spins identical stack to reproduce bug found in staging.

Chapter 12: Deployment to Cloud Platforms [Intermediate]**Chapter focus: Deployment to Cloud Platforms** *(Level: Intermediate)**

Static frontend deploys to CDN (Vercel, Netlify, S3+CloudFront). Node API deploys to Render, Railway, Fly.io, or container orchestrators. Environment secrets configure DATABASE_URL and JWT_SECRET per environment. Run migrations as release step before traffic shifts to new version.

Code Reference:

```
railway up # or: fly deploy
# Ensure DATABASE_URL and NODE_ENV=production set in dashboard
```

What it shows: CLI deploy pushes container; env vars injected by platform secret store.

Topic guide**What it actually is**

Cloud deployment makes full stack apps accessible to users worldwide.

When to use it

Deploy early-even MVP-to validate infra assumptions.

Where to use it

Side projects, client deliverables, and startup MVPs.

Real use example

Founder shares beta URL Friday after CI deploys main branch automatically.

Chapter 13: Error Handling and API Contracts [Intermediate]**Chapter focus: Error Handling and API Contracts** *(Level: Intermediate)**

Consistent error JSON { error: { code, message, details } } helps clients handle failures. Map domain errors to 400/404/409/422; unexpected errors log server-side and return 500 without stack traces. OpenAPI or typed shared packages document contracts between client and server. Version breaking API changes to avoid breaking deployed SPAs.

Code Reference:

```
app.use((err, req, res, next) => {
  console.error(err);
  res.status(err.status || 500).json({ error: { code: err.code || 'INTERNAL', message:
err.publicMessage || 'Server error' } });
});
```

What it shows: Central handler hides internal details while logging full error server-side.

Topic guide**What it actually is**

Consistent API contracts reduce integration bugs between frontend and backend.

When to use it

Define before parallel client/server development on larger features.

Where to use it

Multi-client APIs, mobile plus web, and third-party integrators.

Real use example

Frontend switches on error.code === 'EMAIL_TAKEN' to show field-specific message.

Track: Advanced

Chapter 14: OAuth Social Login Integration [Advanced]**Chapter focus: OAuth Social Login Integration** *(Level: Advanced)**

OAuth2 authorization code flow redirects user to Google/GitHub then returns code to callback URL. Server exchanges code for tokens and creates or links local user record. Store provider subject id; never use access token as session indefinitely. PKCE protects public clients (SPAs) without client secret exposure.

Code Reference:

```
app.get('/auth/google/callback', async (req, res) => {
  const tokens = await exchangeCode(req.query.code);
  const profile = await fetchGoogleProfile(tokens.access_token);
  const user = await upsertUserFromOAuth('google', profile);
  res.redirect(`${CLIENT_URL}/auth/success?token=${signJwt(user)}`);
});
```

What it shows: Callback exchanges code server-side; JWT issued for SPA after OAuth completes.

Topic guide**What it actually is**

OAuth delegates authentication to trusted identity providers users already have.

When to use it

Offer alongside email/password on consumer-facing signup flows.

Where to use it

B2C SaaS, community platforms, and developer tools.

Real use example

User signs up with GitHub in two clicks; avatar and email pre-filled.

Chapter 15: Real-Time Features with WebSockets [Advanced]**Chapter focus: Real-Time Features with WebSockets** (Level: Advanced)**

WebSockets maintain persistent bidirectional connection after HTTP upgrade handshake. ws library on Node broadcasts events to subscribed clients in same room. Heartbeats detect dead connections; clients reconnect with exponential backoff. Authorize socket connections with same JWT verified on HTTP middleware.

Code Reference:

```
wss.on('connection', (socket, req) => {
  const user = verifyTokenFromQuery(req);
  if (!user) return socket.close(4401, 'Unauthorized');
  socket.on('message', (data) => broadcastToRoom(user.orgId, data));
});
```

What it shows: Token verified at connection; messages scoped to user's organization room.

Topic guide**What it actually is**

WebSockets push server updates instantly without client polling overhead.

When to use it

Use for chat, live notifications, collaborative editing, and live dashboards.

Where to use it

Support chat, order tracking, and multiplayer features.

Real use example

Support agent sees customer typing indicator via WebSocket event.

Chapter 16: Redis Caching and Session Store [Advanced]**Chapter focus: Redis Caching and Session Store** *(Level: Advanced)**

Redis in-memory store caches expensive query results with TTL expiration. Cache-aside pattern: read Redis first, on miss query Postgres and populate cache. Invalidate cache keys on writes affecting aggregated data. Redis also stores session data when horizontal scaling Node instances.

Code Reference:

```
const cached = await redis.get('dashboard:${orgId}');
if (cached) return JSON.parse(cached);
const data = await computeDashboard(orgId);
await redis.setex('dashboard:${orgId}', 300, JSON.stringify(data));
return data;
```

What it shows: Five-minute TTL balances freshness vs DB load for dashboard aggregates.

Topic guide**What it actually is**

Redis reduces database load and enables shared session state across servers.

When to use it

Add when read-heavy endpoints slow down or sessions must survive deploys.

Where to use it

Analytics dashboards, rate limiting counters, and pub/sub fan-out.

Real use example

Dashboard load drops DB queries 80% after Redis caches rollup metrics.

Chapter 17: Monorepo Tooling with Turborepo [Advanced]**Chapter focus: Monorepo Tooling with Turborepo** *(Level: Advanced)**

Monorepos share types between packages/apps via workspace protocol. Turborepo caches task outputs-build and test skip unchanged packages. Shared eslint config and tsconfig base reduce duplication. Deploy only changed app while types package version bumps propagate.

Code Reference:

```
{
  "scripts": { "build": "turbo run build" },
  "devDependencies": { "turbo": "^2.0.0" }
```

```
}
```

What it shows: turbo run build executes package dependency graph with remote caching optional.

Topic guide

What it actually is

Monorepo tooling coordinates full stack code in one repository efficiently.

When to use it

Adopt when sharing types/utilities between client and server teams.

Where to use it

Product companies with web app plus admin plus shared UI library.

Real use example

Shared User type changes once; both API and React compile fail until aligned.

Chapter 18: Production Monitoring with Sentry [Advanced]

Chapter focus: Production Monitoring with Sentry** *(Level: Advanced)

Sentry captures frontend and backend exceptions with stack traces and breadcrumbs. Source maps upload makes minified production errors readable. Release tracking ties errors to git SHA for quick rollback decisions. Alert on error rate spikes before users flood support channels.

Code Reference:

```
Sentry.init({ dsn: process.env.SENTRY_DSN, tracesSampleRate: 0.2, release:
process.env.GIT_SHA });
app.use(Sentry.Handlers.requestHandler());
```

What it shows: requestHandler attaches request context; release links errors to deploy version.

Topic guide

What it actually is

Error monitoring surfaces production failures invisible in local development.

When to use it

Integrate before first real users-not after first outage.

Where to use it

Every production web application with SLA commitments.

Real use example

On-call gets Slack alert with stack trace pointing to exact line in release abc123.